

CPU ML Engineering

Five sample tasks

Long-horizon Harbor coding environments.

This document goes over how the folder is structured, what the tasks are, why they are relevant and how many we can make. More in-depth information is given with each sample task in its document.

Pack identification

Field	Overview
Category	CPU ML engineering
Package	Five self-contained Harbor task samples
Task format	Editable workspace, instructions, verifier, reference and alternate solutions, and pass@4 rollouts per task
Source documents	Detailed task information is provided in each accompanying sample task overview

Description

This data product delivers five long-horizon CPU ML-engineering tasks: agentic environments built on real public data and realistic production failure modes. In each, the agent inherits a broken ML system and must do the full engineer's job: investigating the evidence, identifying the underlying defect, repairing the implementation across multiple files and proving the fix against hidden data. Four of the tasks add a gated release service in which the agent deploys, recovers from a staged failure and drives the change to submission; the fifth demands exact reproduction of a production risk service from a formula-free specification. Grading is deterministic and by-value against hidden data, so reward tracks genuine correctness rather than a plausible-looking patch, and shortcut solutions do not score. A pass is real capability signal, not a guess that happened to look right.

The value to a lab is measured, spread difficulty rather than asserted difficulty. Every task is calibrated at pass@4 against two models. On four of the five tasks, Grok 4.7 (xhigh reasoning) lands in a 25–50% band (1–2 of 4 attempts), hard enough to discriminate yet demonstrably solvable, while frontier Claude models pass 4 of 4 (Claude Fable 5 on three tasks, Claude Opus 5 on Condition Register). That 50–75 percentage-point lead is the signal: the tasks separate frontier capability from a strong generalist instead of flooring or saturating both. The fifth, Air Quality Sentinel Rollup, is solved 3 of 4 by both Claude Opus 5 and Grok 4.7, and every miss

from either model is the same single, diagnosable data-quality error: a sharp test of whether an agent checks its assumptions about the data. Difficulty is verifiable, not claimed: every task ships a gold reference solution alongside its grader, together with full rollout traces for both models, so a lab can confirm every result directly. All five share a hardened architecture (protected verifier state, root-owned grading assets, unprivileged agent execution and leak-guarded packaging) that keeps difficulty honest and grading resistant to gaming. The agent container has network access for the harness, and grading runs offline.

Contents

Five task packages, each with full rollout traces for both models and a sample task overview. The overviews document the task, why models failed, the prompt, the environment, grading, and failure, retry and limit rules. Grading is deterministic, behavioral, isolated and control-checked, rewarding genuine correctness rather than plausible patches while resisting shortcut solutions. Beyond the binary reward, every verifier reports each grading component separately, giving a dense diagnostic signal for every attempt.

Folder structure

Each task has its own folder with the same layout:

task/ — the complete Harbor task: instruction, task.toml, environment, verifier and tests, reference and alternate solutions, and the negative-control battery.

<Claude model> pass@4/ — four Claude rollouts (Claude Fable 5, or Claude Opus 5 for Condition Register and Air Quality Sentinel Rollup), with trajectories, verifier output, the aggregate report and build and gate evidence.

Grok 4.7 pass@4/ — four Grok 4.7 (xhigh reasoning) rollouts in the same layout.

<Task> – Overview.docx — the sample task overview.

Technical domains / relevant benchmarks

Designed for frontier AI labs that need difficult, execution-grounded tasks for training and evaluating long-horizon coding and ML agents. The tasks complement Terminal-Bench 2.1, DeepSWE v1.1, SWE-Bench Pro/Verified and MLE-Bench Revised while adding production-ML failure diagnosis and stateful release operations.

Agents must explore unfamiliar repositories, diagnose failures from real data, repair the implementation and prove it against hidden data, in four of the five tasks through a gated deployment workflow, all under partial observability.

The tasks combine DeepSWE- and SWE-bench-style repository engineering, Terminal-Bench-style autonomous terminal execution and MLE-bench-style ML engineering. Their constrained tools, hidden state, ordered operations and end-state verification also exercise capabilities relevant to τ -bench-style stateful tool use, while focusing on autonomous production-ML incidents rather than simulated user conversations.

Capacity

Next 4 weeks	Next 6 months
100 similar-quality tasks	20,000 similar-quality tasks

Human experts

Human task authors and reviewers design, calibrate and review every task, including its reference solutions and control battery.

Raw base data

Public transport, clinical, environmental-sensor and public-funding datasets: Capital Bikeshare hourly demand (UCI Bike Sharing), US airline flight operations (BTS On-Time Performance), UCI Diabetes 130-US hospitals, UCI Air Quality roadside-station sensor logs and USAC E-Rate funding-request records. Exact sources and dataset notes are documented in each sample task overview.

What the world looks like

CPU-only containers with protected verifier state, editable code, real data, hidden validation and, in four of the five tasks, a gated deployment and recovery service. The agent container has network access for the harness, and grading runs offline. Exact environment details are documented per task.

Time, QA and difficulty calibration

Approximately 8–10 hours of authoring, plus additional QA hours, per task. Every task was calibrated at pass@4 with a 9,000-second agent budget per attempt, on the mini-swe-agent harness for both models, with both models run at requested xhigh reasoning effort.

Sample	Pass@4 strict passes	Additional calibration / note
Capital Bike Demand Audit	Claude Fable 5: 4/4 · Grok 4.7 (xhigh reasoning): 1/4	Grok's misses completed the full signed release; the frozen forecast missed the four-month horizon bars.
Fleet Risk Scoring	Claude Fable 5: 4/4 · Grok 4.7 (xhigh reasoning): 1/4	Exact reproduction on an unpublished live window separates close from correct.
Funding Restatement Harness	Claude Fable 5: 4/4 · Grok 4.7 (xhigh reasoning): 1/4	Six of seven fields exact still fails; the label rule has to be probed, not assumed.

Sample	Pass@4 strict passes	Additional calibration / note
Condition Register Carry-Forward	Claude Opus 5: 4/4 · Grok 4.7 (xhigh reasoning): 2/4	Checkpoints certify shape only; the point-in-time register decides the outcome.
Air Quality Sentinel Rollup	Claude Opus 5: 3/4 · Grok 4.7 (xhigh reasoning): 3/4	Every miss from both models masked the well-known -200 marker and missed the log's second one; validation never checks the display.

QA process

Every task passes through a systematic, automated QA pipeline, not an ad-hoc review. A layer of deterministic controls mechanically screens each task for the failure modes that undermine training value: reward-hacking and gameable or forgeable graders, answer leakage and agent-reachable oracles, grading that breaks process isolation, label-proxy features, and false difficulty (a "failure" that was only budget truncation, or a whole batch pinned at an identical floor that signals a broken grader). Each control is two-condition validated: it must fire on a deliberately planted defect and stay silent on a clean task, so the checks are proven rather than assumed.

On top of the static screen, an adversarial control battery executes cheat submissions (do-nothing, output- and ledger-forgery, and task-specific shortcuts) against the real grader in the served environment, and requires every one to score zero, while a gold reference solution scores full marks and structurally different alternate solutions ship with every task (grading is by outcome, not by the shape of one fix). An independent review pass grounds every finding in the actual task files (no fabricated citations), and difficulty is calibrated by running frontier models (Claude and Grok 4.7 at xhigh reasoning) at a fixed pass@4 standard inside the real isolated environment, with validity guards that separate genuine capability failures from budget/truncation artifacts. The pipeline runs automatically and reproducibly; full per-task QA evidence and rollout records accompany each sample.

Verification

Behavior-based grading against hidden data: an environment-owned terminal, a replayed production service or a frozen release, compared with a verifier-owned reference. Four of the five tasks also require operational gates and staged-fault recovery. Grading is fully local and needs no network access.

Per-task deep-dive

Task	What it is about	Deliverable
Capital Bike Demand Audit	A four-month hourly demand forecast frozen by incompatible exports, a weak model and uncalibrated intervals.	Repaired forecast released through a signed, gated walk with staged-fault recovery.
Fleet Risk Scoring	An aircraft grounding-risk service with five modules stripped, rebuilt from a formula-free specification.	Rebuilt service reproducing both the published backtest and an unpublished live run exactly.
Funding Restatement Harness	A perfect local AUC over worthless production scores: a training table built from restated form versions.	Corrected training table released through a receipt-chained, gated walk.
Condition Register Carry-Forward	A readmission export that fans out, with a condition register that is not point-in-time.	Point-in-time register and one-row export, certified and cut over through the release service.
Air Quality Sentinel Rollup	A roadside benzene-alert service that withholds overnight hours and averages the logger's not-recorded markers into its 24-hour display.	Every hour served and a display the station log supports, released through a gated walk with staged-fault recovery.

Example prompts

Each sample's example prompt is quoted in its accompanying sample task overview.

Environment and execution

Common pattern: Python CPU container, editable task workspace, task-specific CLI operations or comparison tools, deterministic verifier, and reference and alternate solutions. Agents run with a 9,000-second budget per attempt. See each sample task overview for exact dependencies, compute limits and commands.

Grading design

Component	Overview
Technical result	Hidden value, metric or terminal compared with a verifier-owned reference.
Operational path	Required action sequence replayed from a protected, root-owned ledger (four of the five tasks).
Recovery	Staged fault must be diagnosed and cleared with the matching recovery family (four of the five tasks).
Isolation	Candidate code runs without access to hidden grader assets or reference outputs.
Reward	Strictly binary per attempt: every condition must hold. The verifier reports each component separately.

How the tasks were created

Public datasets and realistic ML failure modes were screened by measurement, then packaged with reference solutions, alternate positives, negative controls, protected verification and deterministic build checks. Detailed creation notes and supporting artifacts are provided with each sample.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

Name: Suleiman Mahmood	Role: Co-Founder
Signature: /s/Suleiman Mahmood	Date: 9.28.26